

```

1      .data
2      #A, column-major order
3  mat_A: .word 0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15
4      #B: column-major order
5  mat_B: .word 0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15
6      #C: row-major order
7  mat_C: .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
8
9  M:      .word 4 #M=4
10 N:      .word 4 #N=4
11 K:      .word 4 #K=4
12 white:  .asciiiz "_"
13 newL:   .asciiiz "\n"
14
15      .text
16 main:   lw $s0, M($0)
17         lw $s1, N($0)
18         lw $s2, K($0)
19
20         li $t3, 0 #i=0
21         li $t4, 0 #j=0
22         li $t5, 0 #z=0
23
24 loopM:
25
26 loopN:
27     mult $t3, $s1
28     mflo $t6
29     add $t6, $t6, $t4 #C: i * N + j ($t6)
30     sll $t6, $t6, 2
31     li $t2, 0 #Acumulador productos (fila x columna)
32 loopK:
33     mult $t5, $s0
34     mflo $t7
35     add $t7, $t7, $t3 #A: z * M + i ($t7)
36     sll $t7, $t7, 2
37
38     mult $t4, $s2
39     mflo $t8
40     add $t8, $t8, $t5 #B: j * K + z ($t8)
41     sll $t8, $t8, 2

```

```

1      lw    $t0, mat_A($t7) #Carga A
2      lw    $t1, mat_B($t8) #Carga B
3      mult  $t0, $t1        #A[z * M + i] * B[j * K + z]
4      mflo  $t0
5      add   $t2, $t2, $t0
6
7      addi  $t5, $t5, 1      #z++;
8      bne   $t5, $s2, loopK
9      li    $t5, 0
10     sw    $t2, mat_C($t6) #Almacenar fila x columna en C ($t6)
11     addi  $t4, $t4, 1      #j++;
12     bne   $t4, $s1, loopN
13     li    $t4, 0
14     addi  $t3, $t3, 1      #i++;
15     bne   $t3, $s0, loopM
16
17     mult  $s0, $s1        #Muestra los resultados. C-row-major
18     mflo  $s2
19     li    $t0, 0
20     li    $t1, 0
21     li    $t3, 0
22
23 loopP: lw $t2, mat_C($t0)
24     addi  $t0, $t0, 4
25
26     move  $a0, $t2        #I/O syscall: code 1, mostrar entero
27     li    $v0, 1
28     syscall
29
30     la    $a0, white      #I/O syscall: code 4, mostrar string
31     li    $v0, 4
32     syscall
33
34     addi  $t3, $t3, 1
35     bne   $t3, $s0, noNewL
36     li    $t3, 0
37
38     la    $a0, newL       #I/O syscall: code 4, mostrar string
39     li    $v0, 4
40     syscall
41
42 noNewL: addi $t1, $t1, 1
43     bne   $t1, $s2, loopP
44
45 end:
46     li    $v0, 10         #I/O syscall: code 10, salida
47     syscall

```